Bhoomi Sakhrani
2401097
AI&DS - T22
ASD&D

EXPERIMENT 1

Aim: To understand the basics of Git and GitHub, including installation, version control operations, branching, merging, resolving merge conflicts, authentication, and their role in collaborative software development.

LO2: To understand and perform the basic version control operations using Git and GitHub, including repository creation, cloning, file tracking, committing, pushing, pulling, branching, merging, and remote repository management.

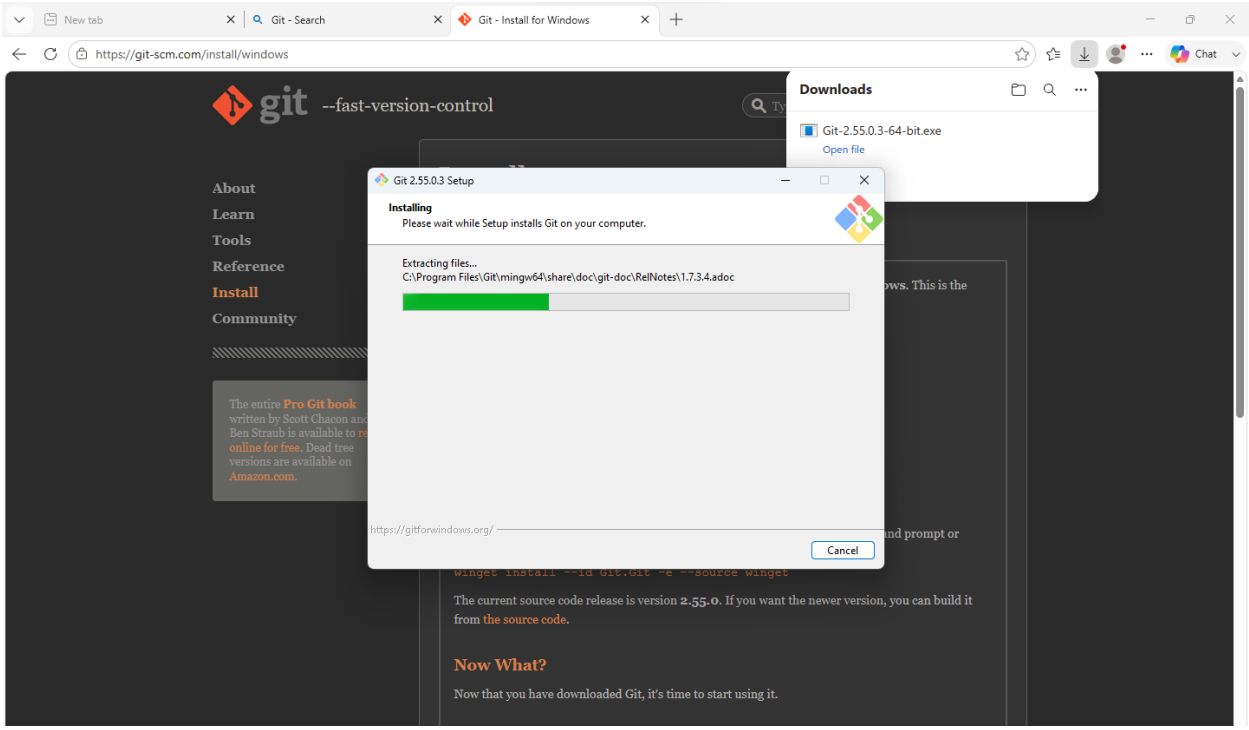
Theory:

Git is a distributed version control system used to track changes in source code, while GitHub is a cloud-based platform that hosts Git repositories and enables collaboration through features such as pull requests, issues, and code reviews.

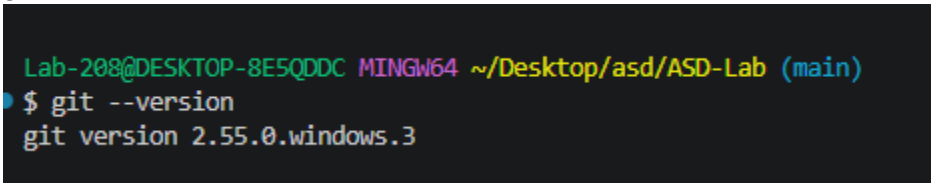
Operations:

1. Installation of Git

Download Git from https://git-scm.com and install it using the default setup. After installation, verify it using the command:



git --version



2. Commit Changes

A commit records changes made to the repository. Typical workflow:

git add .
git commit -m "Initial commit"
Each commit creates a snapshot of the project.

```
PS C:\Users\Lab-208\Desktop\asd\ASD-Lab> git add .
PS C:\Users\Lab-208\Desktop\asd\ASD-Lab> git commit -m "Random maze generator"
[main 6ec895a] Random maze generator
 1 file changed, 28 insertions(+)
 create mode 100644 exp1.py
PS C:\Users\Lab-208\Desktop\asd\ASD-Lab>
```

3. Creation of Branches
Branches allow developers to work on features independently.
git branch feature-branch
git checkout feature-branch
or
git checkout -b feature-branch

```
PS C:\Users\Lab-208\Desktop\asd\ASD-Lab> git checkout -b branch1
Switched to a new branch 'branch1'
PS C:\Users\Lab-208\Desktop\asd\ASD-Lab>
```

4. Merging Branches
Merge combines changes from one branch into another.
git checkout main
git merge feature-branch

```
PS C:\Users\Lab-208\Desktop\asd\ASD-Lab> git add .
PS C:\Users\Lab-208\Desktop\asd\ASD-Lab> git commit -m "calulator"
[branch1 42329be] calulator
 1 file changed, 8 insertions(+)
 create mode 100644 exp1_2.py
PS C:\Users\Lab-208\Desktop\asd\ASD-Lab> git checkout main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)
PS C:\Users\Lab-208\Desktop\asd\ASD-Lab> git merge branch1
Updating 6ec895a..42329be
Fast-forward
 exp1_2.py | 8 ++++++++
 1 file changed, 8 insertions(+)
 create mode 100644 exp1_2.py
PS C:\Users\Lab-208\Desktop\asd\ASD-Lab>
```

5. git switch:

```
PS C:\Users\Lab-208\Desktop\asd\ASD-Lab> git switch branch1
Switched to branch 'branch1'
PS C:\Users\Lab-208\Desktop\asd\ASD-Lab>
```

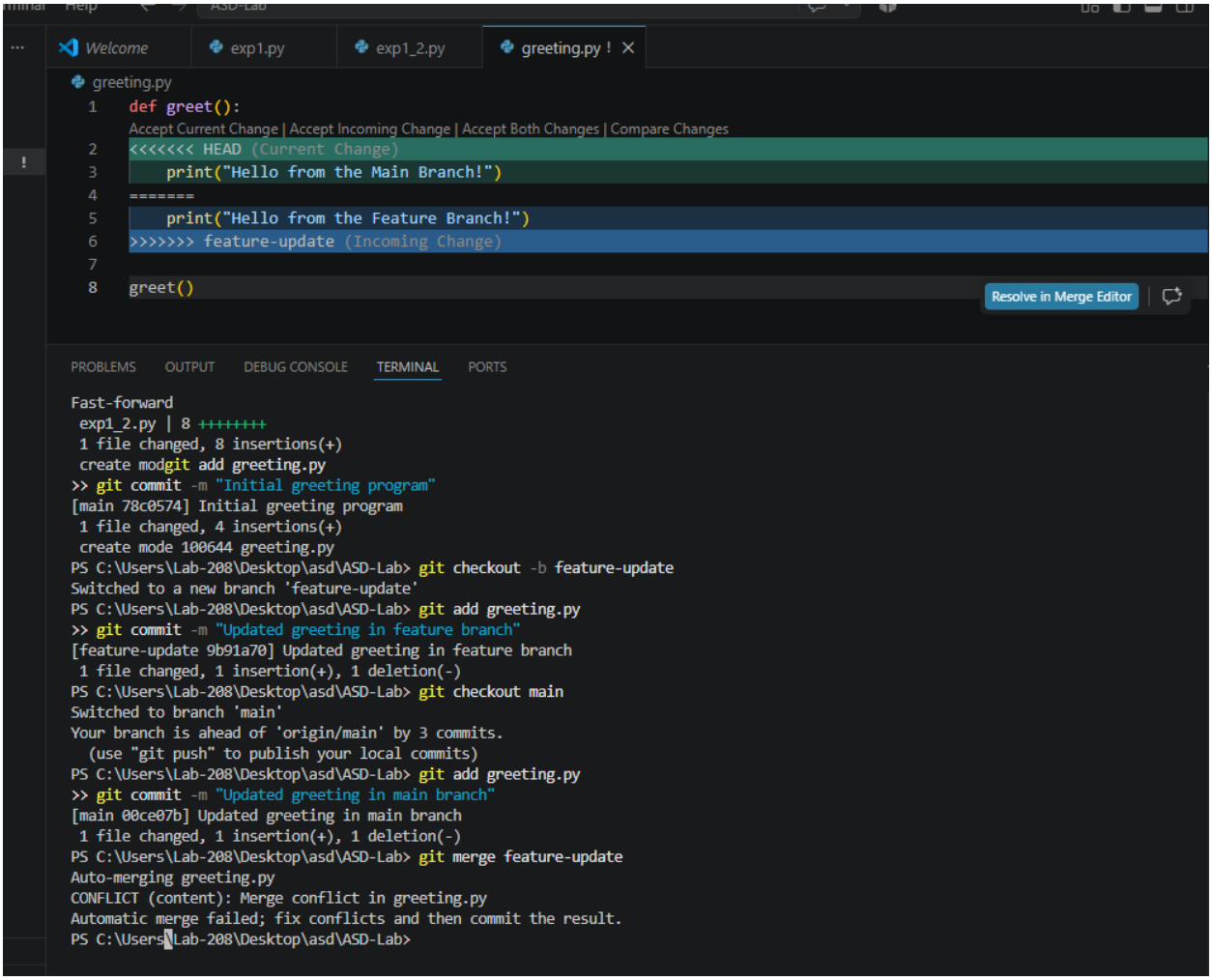
6. Merge Conflicts

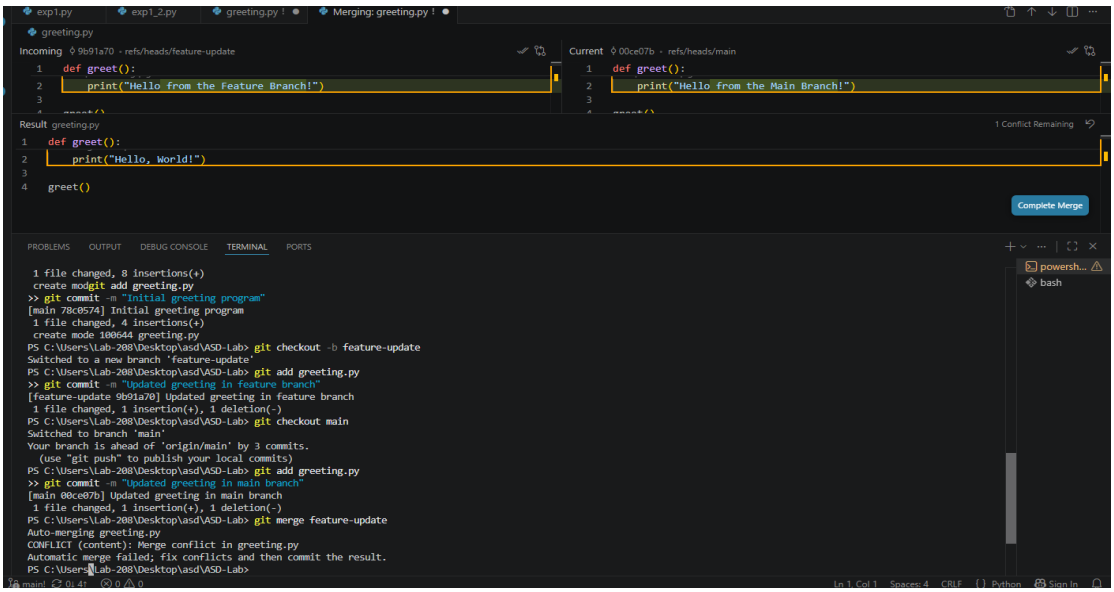
Bhoomi Sakhrani
2401097
AI&DS - T22
ASD&D

Conflicts occur when the same lines of a file are modified in different branches. Git marks conflicting sections, which must be edited manually before completing the merge using:

git add .

git commit





7. Authorization Command

Configure identity and authenticate with GitHub.

git config --global user.name "Your Name"

git config --global user.email "you@example.com"

Modern GitHub authentication typically uses a Personal Access Token (PAT) or SSH keys instead of passwords.

```
PS C:\Users\Lab-208\Desktop\asd\ASD-Lab> git config user.name "bhoomi"
PS C:\Users\Lab-208\Desktop\asd\ASD-Lab> git commit -m "Random maze generator"
Author identity unknown

*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

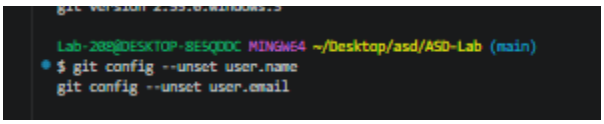
fatal: unable to auto-detect email address (got 'Lab-208@DESKTOP-8E5QDDC.(none)')
PS C:\Users\Lab-208\Desktop\asd\ASD-Lab> git config user.email "bhoomisakhrani25GMAIL.COM"
```

8. Git status

```
PS C:\Users\Lab-208\Desktop\asd\ASD-Lab> git status
On branch branch1
nothing to commit, working tree clean
```

Git unset

Bhoomi Sakhrani
2401097
AI&DS - T22
ASD&D



Conclusion

Git and GitHub simplify version control, enable collaboration, maintain project history, and support safe experimentation using branches and merges. Learning these fundamentals is essential for modern software development.